

Lexer 1

este es el primer elemento que se encuentra en el analisis, el Lexer 1 se encarga del reconocimiento de tokens segun el lenguaje **.form** trabaja con macros y estados

Estados:

YYINITIAL este es el estado por defecto del lexer, este se encarga de reconocer todas las macros definidas excepto las de emojis

STRING: este estado es el encargado de la descomposición de las cadenas de texto, esto con el fin de reconocer a los emojis dentro de las cadenas de texto tambien usa un metodo dentro de la seccion de codigo del usuario para usar un stringBuilder para regresar el string ya formateado y con los emojis normalizados

tambien tiene una lista que se encarga de recolectar todos los errores lexicos, es decir todo lo que no tenga coincidencias con las macros definidas pero esta lista no se le para al parser

Parser 1

este genera el ast para el lenguaje form utilizando varios tipos de nodos dependiendo de la produccion

los mas amplios son los atributos de los estilos y atributos de las tablas ya que estos tiene en su interior listas de estos mismos

Manejo de errores sintácticos: se mantiene una lista **erroresSintacticos** de tipo **ArrayList<ErrorSintactico>** accesible mediante **getErroresSintacticos()**. El método **syntax_error(Symbol s)** se invoca automáticamente por CUP cuando detecta un token inesperado. Este método extrae el lexema, línea y columna del símbolo problemático, y construye un mensaje descriptivo que incluye el token encontrado y la lista de tokens esperados en ese punto de la gramática. Para obtener los tokens esperados utiliza el método **expected_token_ids()** de CUP, traduciendo cada ID numérico a su nombre legible con **syml_name_from_id()**. El método **unrecovered_syntax_error(Symbol s)** se invoca cuando el parser no logra recuperarse de un error; genera un mensaje de "Error irrecuperable" indicando el token cercano al fallo.

Método auxiliar **buildEstilos**: recibe un objeto Atributos y construye un NodoEstilos extrayendo las propiedades de color, color de fondo, familia de fuente, tamaño de texto y borde. Este método se invoca desde múltiples producciones (sección, texto, preguntas) para convertir los atributos de estilo recopilados en un nodo del AST.

Clase auxiliar Atributos

A lo largo de la gramática se utiliza la clase Atributos como un contenedor flexible para recopilar los atributos de cada elemento. Funciona como un mapa clave-valor donde cada producción de atributo individual crea un nuevo Atributos con una sola entrada, y la producción de lista los combina mediante el método **merge()**. Esto permite que los atributos

aparezcan en cualquier orden sin generar conflictos gramaticales. La clase ofrece métodos tipados como `getExpresion()`, `getColor()`, `getString()`, `getListaElementos()`, `getListaExpresiones()`, `getListaFilas()`, `getEstilosMap()` y `toMap()` para extraer los valores con el tipo correcto al momento de construir los nodos del AST.

Gramática de expresiones y precedencia de operadores

Las expresiones se implementan mediante una cascada de no terminales donde cada nivel representa un nivel de precedencia. De menor a mayor precedencia:

expresion delega a **exp_or**. **exp_or** maneja el operador lógico `||` (OR) con asociatividad izquierda. **exp_and** maneja el operador lógico `&&` (AND) con asociatividad izquierda.

exp_relacional maneja los operadores de comparación `>`, `<`, `>=`, `<=`, `==`, `!!` sin asociatividad (no se encadenan, cada comparación es entre dos expresiones aritméticas).

exp_aritmetica maneja suma `+` y resta `-` con asociatividad izquierda. **exp_term** maneja multiplicación `*` y división `/` con asociatividad izquierda. **exp_factor** maneja los operadores unarios: negación aritmética `-`, positivo `+` y negación lógica `~`. **exp_potencia** maneja el operador `^` con asociatividad derecha (la recursión está a la derecha: **exp_mod POTENCIA exp_potencia**). **exp_mod** maneja el operador módulo `%` con asociatividad izquierda.

exp_primaria reconoce los valores iniciales o básicos: **NUMERO** (se convierte a `Double` con `Double.parseDouble()`), **CADENA**, **ID** (se envuelve en **NodoIdentificador**), expresiones entre paréntesis, y el comodín `?` (**NodoComodin.INSTANCE**).

Analizador1 — ejecutor del Lenguaje 1

Descripción general

El Analizador1 es la clase orquestadora que coordina todo el flujo de procesamiento del Lenguaje 1 (archivos **.form**). Recibe el código fuente como un `String` y ejecuta secuencialmente las fases de análisis léxico, sintáctico, semántico y generación de código, retornando como resultado un `String` en formato **.pkm** (Lenguaje 2) o `null` si el análisis falla. Además, centraliza la recolección de los tres tipos de errores (léxicos, sintácticos y semánticos) para que la capa de interfaz pueda consultarlos.

Dependencias

La clase importa y utiliza los siguientes componentes: **Lexer** (analizador léxico generado por JFlex), **Parser** (analizador sintáctico generado por CUP), **EvaluadorExpresiones** (resuelve expresiones del AST a valores concretos), **GeneradorCodigo** (traduce el AST1 a un `String` en formato **.pkm**), **TablaSimbolos** (almacena variables con su tipo y valor), **NodoPrograma** (nodo raíz del AST), y los modelos de error **ErrorLexico** y **ErrorSintactico**.

Estado interno

La clase mantiene tres listas mutables privadas que acumulan los errores de cada fase: erroresLexicos de tipo **MutableList<ErrorLexico>**, erroresSintacticos de tipo **MutableList<ErrorSintactico>**, y erroresSemanticos de tipo **MutableList<String>**. Las tres se limpian al inicio de cada llamada a `analizar()` para garantizar que no se arrastren errores de ejecuciones anteriores.

Paso 1 — Análisis léxico y sintáctico. Se crea una instancia del **Lexer** a partir de un **StringReader** con la entrada, y se pasa al **Parser**. La llamada a **parser.parse()** ejecuta ambos análisis de forma integrada: el parser solicita tokens al lexer bajo demanda. Si el parsing es exitoso, el resultado se extrae como un **NodoPrograma** (la raíz del AST). Todo esto se envuelve en un bloque try-catch-finally: si ocurre una excepción inesperada y no hay errores previos registrados, se agrega un error sintáctico genérico con el mensaje de la excepción. En el bloque finally, siempre se recolectan los errores del lexer y del parser, independientemente de si hubo excepción. Si el AST resulta null (porque el código tenía errores que impidieron construirlo), el método retorna null inmediatamente sin intentar la fase semántica.

Paso 2 — Análisis semántico y generación de código. Si el AST se construyó exitosamente, se instancian los componentes de la fase semántica: una **TablaSimbolos** nueva (vacía), un **EvaludorExpresiones** que recibe la tabla, y un **GeneradorCodigo** que recibe tanto la tabla como el evaluador. La llamada a **generador.generar(ast)** recorre el AST completo, ejecuta las declaraciones de variables (llenando la tabla de símbolos), evalúa las expresiones, ejecuta las estructuras de control (IF, WHILE, FOR, DO-WHILE), resuelve las llamadas a la PokeAPI cuando corresponde, aplica las validaciones semánticas, y produce un String con el código en formato **.pkm**. Los errores semánticos detectados durante este proceso se recolectan del generador y se agregan a la lista interna.

GeneradorEstilos

Recibe un **NodoEstilos** y produce el bloque de etiquetas `<style>...</style>` en formato **.pkm**. Resuelve cada propiedad de estilo (color, background color, font family, text size, border) evaluando las expresiones con el **EvaludorExpresiones** y traduciendo los nodos de color a su representación textual (hex, RGB, HSL o constante). También contiene las validaciones semánticas de rangos RGB (0-255) y HSL (H: 0-360, S: 0-100, L: 0-100).

GeneradorElementos

Recibe un **NodoElemento** y produce la etiqueta **.pkm** correspondiente. Tiene un método **generar()** que despacha según el tipo de elemento: **NodoSeccion** genera `<section=...>`, **NodoTabla** genera `<table>`, **NodoTexto** genera `<open=...>` (el texto se representa como pregunta abierta en el L2), y cada tipo de pregunta genera su etiqueta respectiva (`<open>`, `<drop>`, `<select>`, `<multiple>`). Mantiene contadores internos de secciones y preguntas por tipo que el **GeneradorMetadatos** consulta para el encabezado. También integra el **ServicioPokeApi** para resolver las opciones de `who_is_that_pokemon` y contiene las validaciones semánticas de dimensiones negativas y de índices correctos enteros.

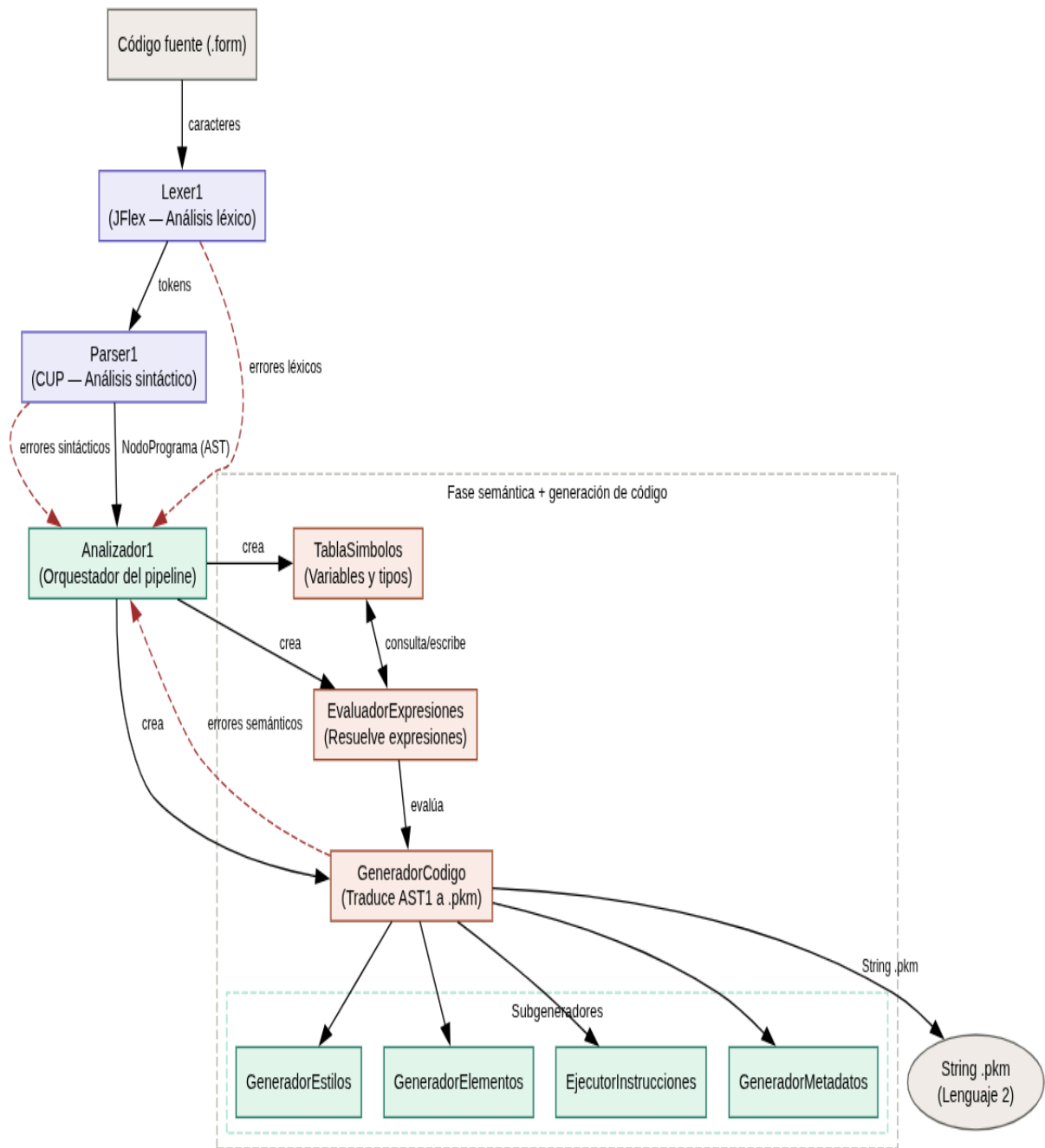
EjecutorInstrucciones

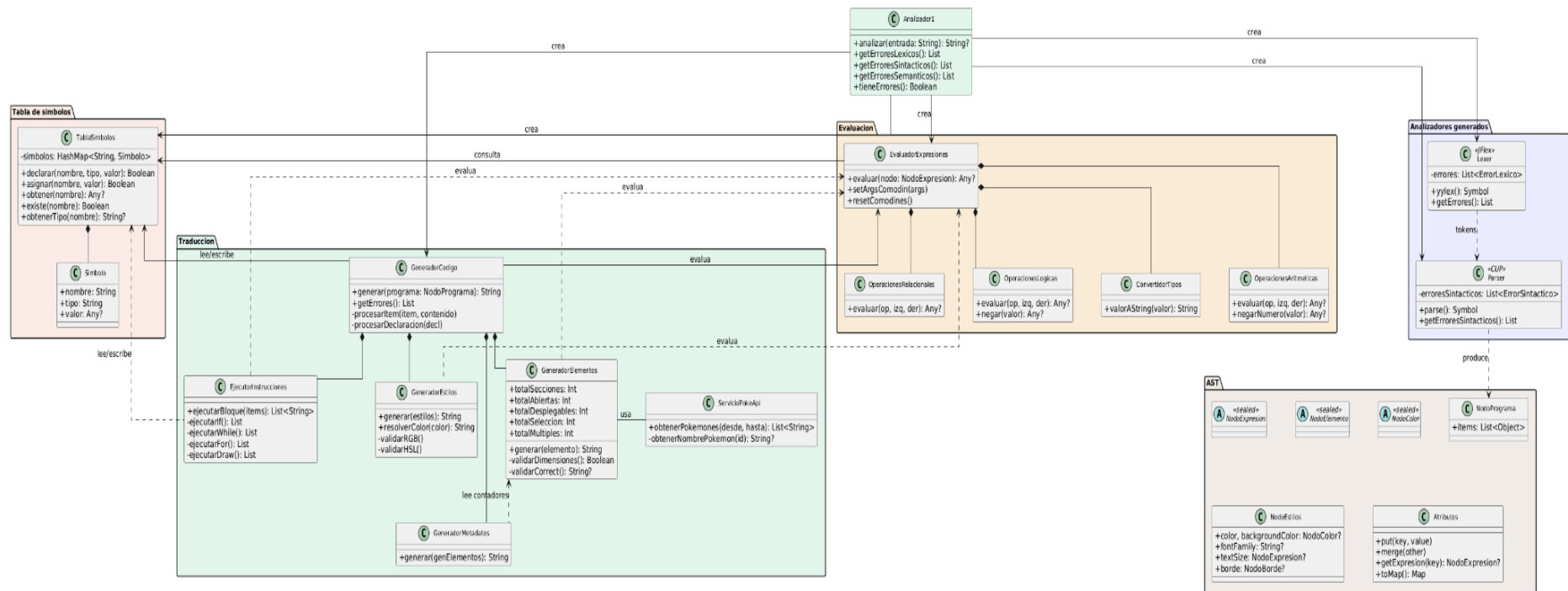
Ejecuta las estructuras de control del lenguaje (IF/ELSE, WHILE, DO-WHILE, FOR clásico, FOR rango) y las llamadas a `draw()`. Recorre los bloques de código evaluando condiciones, iterando ciclos y generando los fragmentos **.pkm** que resultan de cada ejecución.

Implementa un límite por ciclo para prevenir ciclos infinitos. Para **draw()**, configura los comodines en el evaluador, genera el elemento de la variable special y limpia los comodines después. Retorna una lista de Strings con los fragmentos **.pkm** producidos.

GeneradorMetadatos

Produce el encabezado `#### ... ####` del archivo **.pkm** consultando los contadores del **GeneradorElementos**. El encabezado incluye: autor, fecha, hora, descripción, total de secciones, total de preguntas y desglose por tipo (abiertas, desplegadas, selección,múltiples).





Análisis semántico

El análisis semántico del Lenguaje 1 se ejecuta de forma distribuida durante la fase de generación de código. No existe una clase centralizadora de validaciones; en su lugar, cada componente valida las reglas semánticas que corresponden a su contexto. Esta sección cubre los componentes responsables de evaluar las expresiones del AST y detectar errores semánticos relacionados con tipos, operaciones y variables.

ConvertidorTipos

Esta clase actúa como puente entre el sistema de tipos del lenguaje y los tipos nativos de Kotlin. Es utilizada por las tres clases de operaciones para normalizar los operandos antes de operar.

El método **aDouble(valor)** convierte un valor a Double siguiendo estas reglas: si ya es **Double** lo retorna directamente, si es **Int** lo convierte, si es **Boolean** lo traduce a 1.0 (true) o 0.0 (false), y para cualquier otro tipo retorna null indicando que la conversión no es posible. Esta conversión implícita de Boolean a número permite que expresiones como $(x > 5) + 1$ sean válidas en el lenguaje.

El método **aBoolean(valor)** convierte un valor a Boolean: si ya es Boolean lo retorna, si es **Double** lo evalúa como false cuando es 0.0 y true en cualquier otro caso, y para otros tipos retorna null. Esta conversión permite que valores numéricos se usen en contextos lógicos como condiciones de IF o WHILE.

El método **valorAString(valor)** produce la representación textual de cualquier valor para su uso en la generación del .pkm. Tiene un tratamiento especial para Double: si el valor es un entero exacto (por ejemplo 10.0), lo formatea sin decimales ("10" en lugar de "10.0"), lo cual es importante para que los atributos numéricos como width y height aparezcan limpios en el código generado.

EvaluadorExpresiones

Es el componente central de evaluación. Recibe la tabla de símbolos como dependencia y compone internamente las tres clases especializadas de operaciones junto con el convertidor de tipos. Todos comparten la misma lista de errores, de modo que cualquier error semántico detectado en una operación aritmética, relacional o lógica queda registrado en un único punto.

El método **evaluar(nodo)** es recursivo y despacha según el tipo de nodo. Para **NodoNumero** y **NodoCadena** retorna directamente el valor literal. Para **NodoIdentificador** consulta la tabla de símbolos; si la variable no existe genera el error semántico "Variable no ha sido declarada" y retorna null. Para **NodoComodin** consume el siguiente argumento de la lista de comodines configurada por **setArgsComodin**; si no quedan argumentos disponibles genera error y retorna null. Para operaciones binarias y unarias evalúa recursivamente los operandos y delega a la clase de operaciones correspondiente según el operador.

Validaciones semánticas que realiza:

Variable no declarada al evaluar un identificador. Comodines insuficientes al evaluar un ? sin argumentos disponibles. Operador binario desconocido cuando el operador no coincide con ninguno de los reconocidos. Operador unario desconocido para el mismo caso en operaciones de un solo operando.

Propagación de null: cuando cualquier subexpresión retorna null (por un error previo), la expresión contenedora también retorna null sin generar errores adicionales. Esto evita cascadas de mensajes confusos; solo el primer error es el que se reporta.

OperacionesAritmeticas

Maneja las seis operaciones aritméticas del lenguaje y los dos operadores unarios numéricos.

La suma (+) tiene un comportamiento dual: si ambos operandos son convertibles a **Double**, realiza suma numérica. Si al menos uno es **String**, concatena ambos operandos convirtiéndolos a texto con **valorAString**. Si ninguna de las dos condiciones se cumple, genera el error "no se puede sumar". Esta dualidad permite que "Hola " + "mundo" concatene y que 10 + 5 sume numéricamente.

La resta (-), multiplicación (*) y potencia (^) comparten la misma lógica mediante el método privado **aritmetica**: convierten ambos operandos a **Double** y aplican la operación. Si alguno no es convertible, generan error de "operación aritmética inválida".

La división (/) agrega una validación adicional: antes de dividir verifica que el divisor no sea 0.0. Si lo es, genera el error "**División entre cero**" y retorna null. Si los tipos no son compatibles, genera error de "División inválida".

El módulo (%) sigue exactamente el mismo patrón que la división: valida que el divisor no sea cero antes de operar.

La negación unaria (-) convierte el operando a Double y retorna su negativo. Si no es convertible a número, genera error. El positivo unario (+) convierte a Double y retorna el mismo valor; existe para validar que el operando sea numérico.

OperacionesRelacionales

Maneja los seis operadores de comparación del lenguaje, todos retornando **Boolean**.

Los operadores de orden (>, <, >=, <=) solo operan entre valores convertibles a Double. El método privado **comparar** intenta convertir ambos operandos a Double; si alguno no es convertible, genera el error "comparación inválida". Esto significa que comparar un String con un número es un error semántico.

La igualdad (==) tiene un comportamiento más flexible: primero intenta comparar como números (ambos convertibles a Double), luego como Strings (ambos son String). Si ninguna coincide, retorna false sin generar error. La desigualdad (!=) simplemente niega el resultado de la igualdad.

OperacionesLogicas

Maneja los dos operadores lógicos binarios y la negación.

El AND (&&) y el OR (||) convierten ambos operandos a Boolean usando **aBoolean** del convertidor. Si alguno no es convertible, generan el error "Operación lógica inválida".

Gracias a la conversión implícita del **ConvertidorTipos**, valores numéricos como 0.0 (false) y cualquier otro número (true) son válidos en contextos lógicos.

La negación (~) convierte el operando a Boolean y retorna su opuesto. Si no es convertible, genera error.

Resumen de validaciones semánticas del paquete de evaluación

En **EvaluadorExpresiones**: variable no declarada al resolver identificador, comodines insuficientes al resolver ?, operador binario desconocido, operador unario desconocido.

En **OperacionesAritmeticas**: suma entre tipos incompatibles (no numéricos ni String), operación aritmética entre tipos no numéricos, división entre cero, módulo entre cero, negación de un valor no numérico, positivo unario en valor no numérico.

En **OperacionesRelacionales**: comparación de orden entre tipos no numéricos.

En **OperacionesLogicas**: operación lógica entre tipos no convertibles a Boolean, negación de un valor no convertible a Boolean.

Son 12 validaciones semánticas en total dentro de este paquete

com.example.proyecto1_compi1.Logic.evaluacion., todas reportadas a la misma lista de errores del EvaluadorExpresiones.

EjecutorInstrucciones

Ejecuta las estructuras de control y las llamadas a **draw()**. Recibe como dependencias la tabla de símbolos, el evaluador, el generador de elementos y la lista de errores.

Recorrido del AST: el método **ejecutarBloque(instrucciones: List<Any>)** es el punto de entrada. Recorre secuencialmente cada item: si es **NodoElemento** lo delega al

GeneradorElementos, si es **NodoInstruccion** lo despacha mediante **ejecutar(inst)** que resuelve por tipo (**NodoIf**, **NodoWhile**, **NodoFor**, **NodoDraw**, etc.). Los métodos de cada estructura de control pueden llamar recursivamente a **ejecutarBloque** con el cuerpo del bloque, creando el recorrido en profundidad del AST.

El método **esVerdadero(valor)** evalúa condiciones: Boolean directo, Double verdadero si es ≥ 1.0 , cualquier otro tipo es false. Los ciclos (**WHILE**, **DO-WHILE**, **FOR**) tienen un límite de **MAX_ITERACIONES = 100**. El FOR auto-declara la variable iteradora como number si no existe. El draw() configura los comodines en el evaluador, genera el elemento de la variable special y los limpia después.

Validaciones semánticas: variable del FOR no es tipo number. Rango del FOR inválido (null). Límite de iteraciones excedido (WHILE, DO-WHILE, FOR). Variable de draw no inicializada. Variable de draw no es tipo special. Variable ya declarada con tipo diferente al reasignar.

GeneradorElementos

Traduce cada **NodoElemento del AST a su etiqueta .pkm** correspondiente. Mantiene contadores de secciones y preguntas por tipo. Compone internamente el **ServicioPokeApi**.

Recorrido del AST: el método **generar(elemento: NodoElemento)** despacha según el tipo:

NodoSeccion a **generarSeccion**, **NodoTabla** a **generarTabla**, **NodoTexto** a **generarTexto**, y cada tipo de pregunta a su método respectivo. **generarSeccion** y **generarTabla** llaman recursivamente a **generar()** para sus elementos hijos, creando el recorrido en profundidad de la jerarquía visual. Las preguntas con **pokemonDesde/pokemonHasta** invocan al **ServicioPokeApi** para resolver las opciones.

El método **validarCorrect** verifica que el índice sea entero: decimales con .0 se truncan ($1.0 \rightarrow 1$), decimales no enteros (1.5) generan error y se descartan. El método

validarDimensiones recorre los atributos de un mapa y **validarDimensionDirecta** valida un atributo individual; ambos verifican que no sean negativos. **validarDimensionDirecta** además detecta comodines con **contieneComodin** (recursivo) y omite la validación en ese caso.

Validaciones semánticas: dimensiones negativas en secciones, tablas, textos y preguntas (width, height, pointX, pointY). Índice de respuesta correcta no entero. Errores de PokeAPI propagados.

GeneradorEstilos

Traduce un **NodoEstilos al bloque <style>...</style> del .pkm**.

Recorrido del AST: el método **generar(estilos: NodoEstilos?)** recorre cada propiedad del nodo (**color**, **backgroundColor**, **fontFamily**, **textSize**, **borde**) y genera la etiqueta

correspondiente. El método `resolverColor(color: NodoColor)` despacha según el tipo de color (Hex, HSL, RGB, Nombre) y aplica las validaciones de rango cuando corresponde. Validaciones semánticas: RGB con componente R, G o B fuera de rango (0-255). HSL con H fuera de rango (0-360) o S/L fuera de rango (0-100).

GeneradorMetadatos

Produce el encabezado `### ... ###` del .pkm. El método `generar(genElementos, autor)` consulta los contadores del **GeneradorElementos** y genera los campos: autor, fecha, hora, descripción, total de secciones, total de preguntas y desglose por tipo. No recorre el AST ni realiza validaciones semánticas.

resumen de validaciones semanticas

Reglas semánticas del proyecto

Variable ya fue declarada (redeclaración) → **TablaSimbolos**
Variable no ha sido declarada (al asignar) → **TablaSimbolos**
Variable no ha sido declarada (al obtener) → **TablaSimbolos**
Tipo incompatible en asignación (number ← string o viceversa) → **TablaSimbolos**
Variable no declarada al resolver identificador en expresión → **EvaluadorExpresiones**
Comodines insuficientes al resolver ? → **EvaluadorExpresiones**
Operador binario desconocido → **EvaluadorExpresiones**
Operador unario desconocido → **EvaluadorExpresiones**
Suma entre tipos incompatibles (no numéricos ni String) → **OperacionesAritmeticas**
Operación aritmética entre tipos no numéricos (-, *, ^) → **OperacionesAritmeticas**
División entre cero → **OperacionesAritmeticas**
Módulo entre cero → **OperacionesAritmeticas**
Negación unaria de un valor no numérico → **OperacionesAritmeticas**
Positivo unario en valor no numérico → **OperacionesAritmeticas**
Comparación de orden entre tipos no numéricos (>, <, >=, <=) → **OperacionesRelacionales**
Operación lógica entre tipos no convertibles a Boolean (&&, ||) → **OperacionesLogicas**
Negación lógica (~) de valor no convertible a Boolean → **OperacionesLogicas**
Variable del FOR no es tipo number → **EjecutorInstrucciones**
Rango del FOR inválido (null) → **EjecutorInstrucciones**
Límite de iteraciones excedido en WHILE → **EjecutorInstrucciones**
Límite de iteraciones excedido en DO-WHILE → **EjecutorInstrucciones**
Límite de iteraciones excedido en FOR → **EjecutorInstrucciones**
Variable de draw no inicializada o no existe → **EjecutorInstrucciones**
Variable de draw no es de tipo special → **EjecutorInstrucciones**
Variable ya declarada con tipo diferente (reasignación en bloque) → **EjecutorInstrucciones**
Dimensiones negativas (width, height, pointX, pointY) en secciones, tablas, textos y preguntas → **GeneradorElementos**
Índice de respuesta correcta no es entero (correct debe ser entero o .0) → **GeneradorElementos**
Componente RGB fuera de rango (R, G o B fuera de 0-255) → **GeneradorEstilos**
Componente HSL fuera de rango (H fuera de 0-360, S o L fuera de 0-100) → **GeneradorEstilos**
Error al consultar la PokeAPI (pokémon no encontrado) → **ServicioPokeApi**

